

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

**Тема: РАЗРАБОТКА ПРОГРАММЫ ДЛЯ ВИЗУАЛИЗАЦИИ
АЛГОРИТМА КРАСКАЛА НА ЯЗЫКЕ ПРОГРАММИРОВАНИЯ
KOTLIN**

Студент гр. 4382

Шлей Н.А.

Студент гр. 4382

Исаев А.В.

Руководитель

Фирсов М.А.

Санкт-Петербург
2026

ЗАДАНИЕ

НА УЧЕБНУЮ ПРАКТИКУ

Студент: Шлей Н.А.

Группа: 4382

Студент: Исаев А.В.

Группа: 4382

Тема практики: Разработка программы для визуализации алгоритма Краскала на языке программирования Kotlin

Задание на практику:

Командная поэтапная разработка программы визуализации алгоритмов на Kotlin с графическим интерфейсом.

Алгоритм: Краскал

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 17.07.2026

Дата защиты отчета: 17.07.2026

Студент

Шлей Н.А.

Студент

Исаев А.В.

Руководитель

Фирсов М.А.

АННОТАЦИЯ

В рамках учебной практики разработано приложение на Kotlin + JavaFX для визуализации алгоритма Краскала построения минимального остовного дерева во взвешенных неориентированных графах. Реализованы алгоритм на основе DSU, пошаговый режим с двухфазной цветовой подсветкой рёбер (жёлтый — рассматриваемое, зелёный — добавлено, красный — отклонено), интерактивное создание графа, загрузка из файла, автозапуск с регулировкой скорости, откат и сброс. Программа корректно обрабатывает несвязные графы, предотвращает петли и кратные рёбра, устойчива к некорректному вводу. Проект позволил закрепить навыки ООП, работы с JavaFX и реализации алгоритмов на графах.

SUMMARY

An application was developed using Kotlin + JavaFX for visualizing Kruskal's algorithm for minimum spanning tree in weighted undirected graphs. The algorithm uses DSU, step-by-step mode with two-phase edge highlighting (yellow — current, green — added, red — rejected), interactive graph creation, file loading, auto-play with speed control, undo, and reset. The program handles disconnected graphs (spanning forest), prevents self-loops and multiple edges, and is robust to invalid input. The project reinforced skills in OOP, JavaFX, and graph algorithms.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 ПОСТАНОВКА ЗАДАЧИ.....	6
1.1. Предметная область	6
1.2. Задачи практики	8
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	10
2.1. Вводное задание	10
2.2. Составление спецификации программы	10
2.2.1. Требования к вводу исходных данных	20
2.2.2. Требования к визуализации	20
2.2.3. Уточнение требований после сдачи прототипа	21
2.2.4. Уточнение требований после сдачи альфа-версии	21
2.2.5. Уточнение требований после сдачи бета-версии.....	22
2.3. Командная итеративная разработка приложения	22
2.4. Структуры данных и их методы.....	22
2.5. Тестирование	30
2.5.1. Тестирование интерфейса	30
2.5.2. Тестирование алгоритма	30
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ.....	32
4 ОТЗЫВ РУКОВОДИТЕЛЯ.....	34
ЗАКЛЮЧЕНИЕ.....	35
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	36
ПРИЛОЖЕНИЕ А.....	37

ВВЕДЕНИЕ

Современная разработка программного обеспечения требует не только глубокого понимания алгоритмов и структур данных, но и умения воплощать алгоритмические идеи в полноценные пользовательские приложения с интуитивно понятным интерфейсом. Учебная практика по направлению «Новые языки программирования» была посвящена освоению языка Kotlin и применению его средств для командной разработки визуализатора алгоритма построения минимального остовного дерева. В качестве предметной области была выбрана визуализация работы алгоритма Краскала на взвешенных неориентированных графах.

Алгоритм Краскала является классическим жадным алгоритмом, широко применяемым в задачах проектирования сетей (транспортных, телекоммуникационных, энергетических), кластеризации данных, построения остовных деревьев и решения задачи о соединении всех точек с минимальной суммарной стоимостью. Графическое представление работы алгоритма позволяет не только увидеть итоговое минимальное остовное дерево, но и проследить пошаговую логику принятия решений, понять, почему конкретное ребро включается в остов или отвергается из-за образования цикла, а также оценить работу структуры данных «система непересекающихся множеств» (DSU).

Целью практики являлось освоение основных средств языка Kotlin и разработка законченного графического приложения, демонстрирующего алгоритм Краскала. Для достижения цели требовалось выполнить индивидуальное вводное задание по изучению языка программирования Kotlin, подготовить и согласовать спецификацию мини-проекта, распределить обязанности между участниками бригады, реализовать приложение в нескольких последовательных версиях (прототип, альфа, бета, релиз), организовать работу в общем репозитории Git и провести тестирование полученного результата.

1 ПОСТАНОВКА ЗАДАЧИ

1.1. Предметная область

Взвешенный неориентированный граф состоит из множества вершин и множества рёбер, каждому из которых присвоен числовой вес. Ребро соединяет две вершины, при этом порядок их указания не имеет значения. Задача построения минимального остовного дерева (МОД) заключается в выборе подмножества рёбер, соединяющего все вершины графа, с минимальной суммарной стоимостью (весом) и без образования циклов. Такое дерево находит применение в проектировании транспортных сетей, телекоммуникационных соединений, электрических сетей и других инфраструктурных задачах, где требуется соединить все объекты с минимальными затратами.

Одним из эффективных алгоритмов решения данной задачи является алгоритм Краскала, основанный на жадном подходе. На каждом шаге алгоритм выбирает ребро с наименьшим весом среди ещё не рассмотренных. Если добавление этого ребра не создаёт цикла в текущем остове, оно включается в решение. Для проверки образования циклов используется структура данных «система непересекающихся множеств» (DSU, CHM), которая позволяет эффективно определять, принадлежат ли две вершины одному и тому же компоненту связности. Алгоритм завершается, когда в остов добавлено $(V-1)$ ребро (для связного графа) или когда все рёбра рассмотрены (для несвязного графа в этом случае строится остовный лес).

Визуализация алгоритма должна показывать состояние графа после каждого логического шага: текущее рассматриваемое ребро, результат проверки на цикл, уже добавленные в остов рёбра и отклонённые. Пользователю необходимо видеть цветовую индикацию рёбер (серый — ожидает, жёлтый — рассматривается, зелёный — добавлен, красный — отклонён), текущий вес построенного остова и количество добавленных рёбер. Такое представление делает алгоритм понятнее и позволяет использовать программу как учебное средство при изучении жадных алгоритмов и структур

данных.

Приложение ориентировано на графы умеренного размера, которые удобно анализировать визуально. Для больших графов предусмотрена прокрутка холста и мини-карта для обзорного отображения. Все вершины имеют целочисленные идентификаторы, рёбра подписаны весами, а некорректные операции — добавление петли (ребра, соединяющего вершину с самой собой), кратного ребра или ребра с недопустимым весом — должны отклоняться с выводом понятного сообщения об ошибке.

1.2. Задачи практики

Для достижения поставленной цели были определены следующие задачи:

- Изучить язык Kotlin, его объектно-ориентированные возможности, коллекции, обработку исключений и средства ввода-вывода, а также освоить библиотеку JavaFX для построения графических интерфейсов;
- Выполнить индивидуальное вводное задание (курс на Stepik);
- Составить спецификацию визуализатора алгоритма Краскала, определить формат входных данных (количество вершин, рёбер, тройки "начальная вершина — конечная вершина — вес") и установить ограничения (запрет петель и кратных рёбер);
- Разработать план итеративной реализации с поэтапной сдачей прототипа, альфа-, бета- и финальной версий, согласовав контрольные точки с руководителем;
- Организовать совместную работу бригады в репозитории Git, распределить зоны ответственности между backend- и frontend-разработчиками;
- Реализовать модель взвешенного неориентированного графа (классы Vertex, Edge, Graph) и алгоритм Краскала на основе системы непересекающихся множеств (DSU) с проверкой образования циклов и построением минимального остовного дерева или леса для несвязных графов;
- Создать графический интерфейс на JavaFX, обеспечивающий интерактивное редактирование графа (добавление вершин по клику, создание рёбер с указанием веса, удаление элементов) и отображение с поддержкой масштабирования и мини-карты;
- Реализовать пошаговый режим визуализации с двухфазной подсветкой рёбер (жёлтый — текущее рассматриваемое, зелёный — добавлено в MST, красный — отклонено), а также функции отката к предыдущему шагу и автоматического воспроизведения с регулируемой скоростью;
- Реализовать загрузку графа из текстового файла с валидацией данных и

сохранение текущего графа в файл, обеспечить понятную обработку ошибок ввода-вывода и некорректного формата;

- Провести функциональное тестирование алгоритма на графах различной структуры (связные, несвязные, с петлями и кратными рёбрами), проверить корректность работы интерфейса во всех режимах и устойчивость к ошибочным действиям пользователя;
- Собрать приложение в исполнимый JAR-файл, проверить его запуск вне среды разработки и подготовить отчётную документацию по итогам практики.

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Вводное задание

На начальном этапе практики был пройден курс на Stepik по языку программированию Kotlin[3].

2.2. Составление спецификации программы

Название проекта: Программный комплекс для пошаговой интерактивной визуализации алгоритма Краскала.

Авторы: Шлей Никита, Исаев Андрей

1. Введение

1.1. Назначение программы

Программное обеспечение предназначено для наглядной демонстрации работы жадного алгоритма Краскала при построении МОД во взвешенном неориентированном графе.

1.2. Область применения

Инструмент ориентирован на использование в образовательных целях студентами и преподавателями в рамках дисциплин «Алгоритмы и структуры данных» для глубокого понимания работы Системы Непересекающихся Множеств (СНМ) и теории графов.

1.3. Целевая аудитория

- 1) Студенты, изучающие алгоритмы и структуры данных
- 2) Преподаватели для демонстрации работы алгоритма Краскала
- 3) Разработчики, изучающие Kotlin и визуализацию алгоритмов

1.4. Термины и определения

Термин	Определение
Граф	Совокупность вершин и соединяющих их ребер
Вершина	Базовый элемент, точка пересечения или объект, между которыми устанавливаются связи (рёбра)
Ребро	Связь между двумя вершинами, имеющая вес

Вес ребра	Числовая характеристика ребра (стоимость, расстояние)
Минимальное остовное дерево (МОД)	Остовное дерево графа, имеющее минимальный возможный вес, где под весом дерева понимается сумма весов входящих в него рёбер
Алгоритм Краскала	Жадный алгоритм построения MST, сортирующий ребра по весу и добавляющий их, если они не образуют цикл
CHM (DSU)	Система непересекающихся множеств — структура данных для проверки принадлежности элементов к одному множеству
Визуализация	Графическое представление работы алгоритма с подсветкой шагов
Пошаговый режим	Выполнение алгоритма по одному шагу с задержкой или по кнопке
Интерактивность	Возможность взаимодействия пользователя с графом (добавление(/удаление) вершин(/ребер)

Таблица 1 – Терминология с определениями

1.5. Список сокращений

Сокращение	Расшифровка
МОД	Минимальное Остовное Дерево
CHM	Система Непересекающихся Множеств
MST	Minimum Spanning Tree
DSU	Disjoint Set Union
UI	User Interface (Пользовательский интерфейс)
GUI	Graphical User Interface (Графический пользовательский

	интерфейс)
IDE	Integrated Development Environment (Интегрированная среда разработки)

Таблица 2 - Сокращения

2. Технический стек и ограничения

2.1. Инструментарий проекта

Компоненты	Технология
Язык разработки	Kotlin 1.9+
Графический фреймворк	JavaFX или Swing
Система сборки	Gradle
Репозиторий	GitHub
Версия Java	21
Формат сборки	.jar (исполнимый файл)

Таблица 3 – Инструменты

2.2. Архитектура

Разделение на независимый логический модуль (backend) и графический интерфейс (frontend) на базе стандартных графических библиотек JVM (Swing/JavaFX).

2.3. Ограничение

Исполнимый файл должен работать вне IDE

3. Ожидаемые функциональные возможности

3.1. Модуль интерактивного ввода и управления графом

3.1.1. Редактор холста

Редактор холста предоставляет пользователю интуитивно понятный способ создания графа без необходимости писать код или работать с файлами. Весь процесс построения графа выполняется с помощью мыши и клавиатуры.

Интерактивное добавление вершин

Клик левой кнопкой мыши по свободному месту на холсте. В точке клика

появляется вершина с уникальным номером (ID). Каждая новая вершина получает следующий по порядку номер (0, 1, 2, ...).

Визуальное представление:

- 1) Вершина отображается как закрашенный круг.
- 2) Внутри круга её номер.
- 3) При наведении курсора на вершину она должна подсвечиваться

Проведение взвешенных ребер

Пользователь может последовательно кликать по двум вершинам, выделяя их, и между ними образуется ребро. После этого появляется поле для ввода веса ребра (целое число).

Визуальное представление:

- 1) Ребро отображается как линия между вершинами
- 2) над ребром отображается его вес
- 3) Возможность редактирования веса после создания

Также альтернативно реализация drag-and-drope - возможность пользователю соединить вершины путем перетаскивания вершины 1 в вершину 2

3.1.2. Импорт файлов

- 1) Возможность загрузки структуры графа из текстового файла конфигурации
- 2) Поддержка формата: „[ID вершины1] [ID вершины2] [Вес]“

3.1.3. Управление состоянием

- 1) Кнопка полной очистки холста для построения новой конфигурации графа
- 2) Сохранение текущего графа в файл

3.2. Модуль пошаговой визуализации и симуляции

3.2.1. Дискретное управление

- 1) Шаг вперед — выполнение одной итерации алгоритма
- 2) Шаг назад — откат к предыдущему состоянию

3.2.2. Автоматическая симуляция

- 1) Режим непрерывного воспроизведения (Auto-play)

- 2) Возможность постановки на паузу
- 3) Возможность настройки скорости воспроизведения (ползунок)

3.2.3. Анимационная индикация(Предварительно)

Цвет	Значение
Зеленый	Ребро успешно добавлено в каркас МОД
Желтый	Текущее рассматриваемое ребро на данном шаге итерации
Красный	Ребро не добавлено в каркас МОД (создает цикл)
Серый	Ребра, ожидающие очереди проверки

Таблица 4 – Цветовое кодирование ребер

3.2.4. Работа программы (Алгоритм Краскала)

Идея:

Алгоритм Краскала ищет самый дешевый способ соединить все вершины так, чтобы между ними не образовались циклы. Для этого программа делает следующие шаги:

Шаг А. Сортировка

Как только пользователь нажимает кнопку запуска, программа сортирует все ребра графа по возрастанию веса.

Шаг В. Основной цикл

Программа берет из начала очереди первое (самое дешевое) ребро, подсвечивает его желтым цветом(?) на экране и проверяет:

Случай 1 Цикла нет): Если эти две вершины еще ни разу не были соединены друг с другом (даже обходными путями через другие точки), алгоритм одобряет это ребро. Оно красится в зеленый цвет(?), добавляется в итоговое дерево, а в панель логов пишется оповещение об успешном добавлении ребра.

Сценарий 2 (Образуется цикл): Если к этим двум точкам уже можно добраться по другим зеленым линиям, то проводить между ними еще одну прямую линию нельзя, т.к получится цикл. Алгоритм отклоняет это ребро, оно

красится в красный цвет(?), а в log пишется оповещение о невозможности соединения ребер.

Шаг С.

Программа повторяет Шаг В для каждого следующего ребра в очереди до тех пор, пока все точки на холсте не окажутся связаны в единую сеть. Как только это произойдет, алгоритм завершит работу и на холсте останется МОД.

Сложность: $O(n \log(n))$

3.3. Информационный логгер

3.3.1. Текстовое сопровождение

Динамический вывод пояснений к каждому шагу алгоритма в специализированную текстовую панель для детального анализа работы СНМ на каждом шаге.

3.3.2. Информационная статистика

- 1) Текущий вес MST
- 2) Количество добавленных ребер
- 3) Номер текущего шага / общее количество шагов

4. Формат входных конфигурационных файлов

4.1. Структура файла

Входной файл импорта должен соответствовать строгому шаблону:

Строка 1: „[Количество вершин V] [Количество ребер E]“

Строки от 2 до E+1: „[ID начальной вершины] [ID конечной вершины] [Вес ребра (целое число)]“

4.2. Пример правильного файла конфигурации

4 4

1 2 5

2 3 3

3 4 6

4 1 4

5. Требования к пользовательскому интерфейсу

5.1 Макет окна (рис. 1)

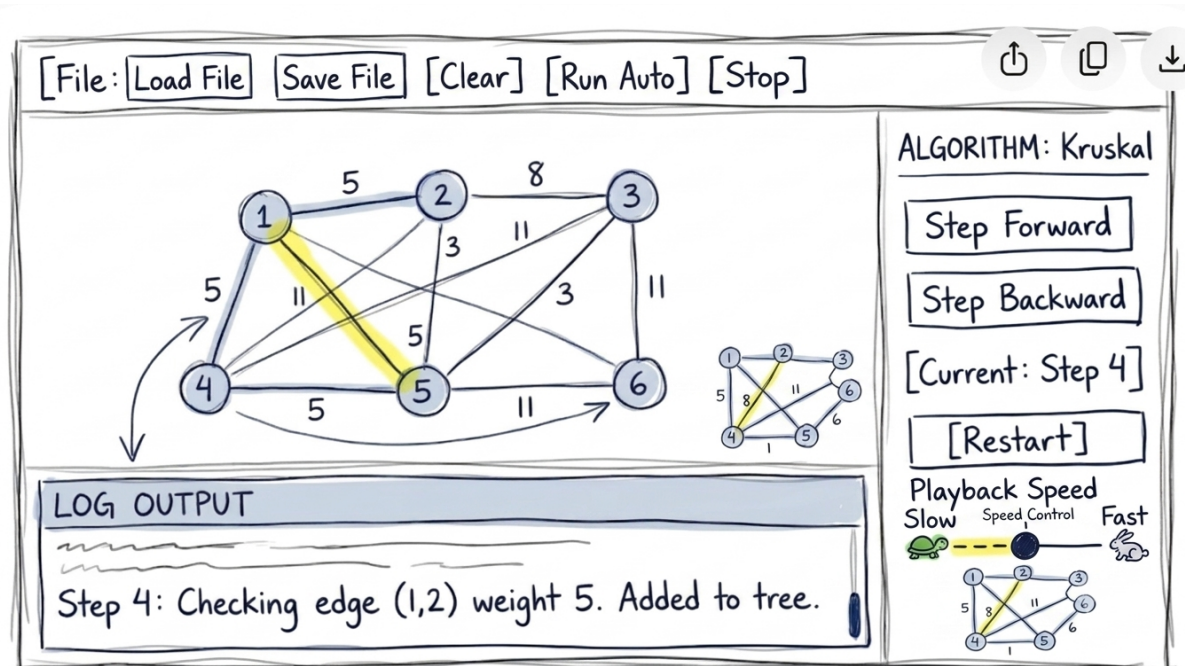


Рисунок 1 – Макет окна

5.2 Непосредственно требования

- 1) Окно должно быть масштабируемым
- 2) Холст должен занимать основную часть окна
- 3) Все кнопки должны иметь подписи
- 4) Поддержка русского языка в интерфейсе

6. Нефункциональные требования

6.1. Надежность

- 1) Приложение корректно обрабатывает ошибки ввода: неверный формат файла, пустой граф, несвязный граф.
- 2) Все ошибки выводятся пользователю в понятном виде (диалоговые окна или текстовые сообщения в интерфейсе).
- 3) Некорректные действия пользователя блокируются (например, запуск алгоритма без графа).

6.2. Кроссплатформенность

- 1) Поддерживаемые ОС: Windows 10/11, Linux (Ubuntu 20.04+, Debian 11+), macOS (12+).
- 2) Для запуска требуется JVM версии 21 и установленное графическое окружение.

6.3. Документация

- 1) Спецификация требований (данный документ).
- 2) Инструкция пользователя (в составе отчета по практике).
- 3) Исходный код снабжен понятными комментариями.

7. План разработки

1-я неделя:

7 июля(вт) - формирование темы проекта и выбор группы

9 июля(чт) - защита вводного задания + демонстрация Git репозитория и спецификации

10 июля(пт) - итоговая сдача вводного задания и согласование спецификации.

Начало работы над прототипом

11 июля (сб) - предоставление прототипа преподавателю - главное окно с холстом и панелью управления.

По окончании работы над прототипом проекта должны быть представлены:

- 1) Кнопки: «Загрузить», «Сохранить», «Очистить», «Запустить», «Шаг», «Сброс».
- 2) Загрузка графа из файла и отображение его на холсте.
- 3) Базовые классы: Vertex, Edge, Graph.
- 4) Сборка проекта в .jar.

12 июля (вс) - внесение правок в проект прототипа (по необходимости). Иначе работа над первой версией проекта

По окончании работы над первой версией проекта должны быть представлены:

- 1) Полноценная работа алгоритма Краскала
- 2) Вывод результата в текстовом виде (список рёбер MST, вес).
- 3) Текстовый log работы алгоритма: последовательность ребер с указанием результата рассмотрения для каждого
- 4) Отображение финального MST на холсте.
- 5) Обработка ошибок: пустой граф, несвязный граф, неверный файл.

2-я неделя:

13 июля (пн) - крайний срок сдачи прототипа (по необходимости). Сдача первой версии проекта (по возможности)

14/15 июля (ср-чт) — внесение правок и финальная сдача первой версии (по необходимости). Сдача второй версии проекта(по возможности)

По окончании работы над второй версией проекта должны быть представлены:

- 1) Пошаговый режим (кнопки «Вперёд» / «Назад»).
- 2) Подсветка рёбер: жёлтое (текущее), зелёное (добавлено), красное (отклонено).
- 3) Текстовые пояснения к каждому шагу.
- 4) Автозапуск с паузой.
- 5) Счётчики: вес MST, количество рёбер, номер шага.

16 - 17 июля (пт) — сдача второй версии проекта (по необходимости). Работа над отчетом. Сдача финальной версии проекта и отчета.

- 1) Интерактивное создание графа мышью (вершины по клику, рёбра по клику по двум вершинам).
- 2) Сохранение графа в файл.
- 3) Кнопка «Новый граф» (очистка).
- 4) Удаление вершин и рёбер.
- 5) Итоговая сборка .jar и проверка запуска вне IDE.
- 6) Отчёт по практике.

18 июля (сб) - в случае необходимости сдача финальной версии и отчета (по согласованию с преподавателем)

3-я неделя

20 июля (пн) - аналогично 18 июля

8. Тестирование

Тестирование проводится на каждом этапе разработки.

Основные предполагаемые проверки:

- 1) Корректность алгоритма — сравнение результата работы программы с ожидаемым MST на тестовых графах.

- 2) Работа интерфейса — проверка кнопок, отображения графа, пошагового режима, анимации.
- 3) Загрузка и сохранение файлов — корректность parsinga и записи данных.
- 4) Обработка ошибок — программа не падает при неверном вводе, показывает понятные сообщения.

9. Распределение ролей

Шлей Никита - backend разработка

Задачи:

- 1) Разработка алгоритма краскала
- 2) Реализация структуры СНМ для проверки циклов
- 3) Обработка загрузки графа из файла: чтение, parsing
- 4) Экспорт графа
- 5) Реализация логики
- 6) Управление состояниями: переходы вперед/назад и т.д
- 7) Обработка ошибок
- 8) Сборка .jar файла

Исаев Андрей - frontend разработка

- 1) Разработка главного окна приложения (холст, панели управления, информационные поля).
- 2) Отрисовка графа на холсте
- 3) Реализация интерактивного создания графа
- 4) Подсветка рёбер в процессе работы алгоритма (зелёный, жёлтый, красный, серый).
- 5) Реализация анимации
- 6) Текстовое сопровождение
- 7) Обновление статистики
- 8) Обработка событий интерфейса

Совместно

- 1) Согласование формата данных между backend и frontend

- 2) Отладка
- 3) Подготовка отчета по практике и демонстрация работы перед преподавателем

2.2.1. Требования к вводу исходных данных

Входные данные могут быть заданы пользователем интерактивно с помощью мыши на холсте или загружены из текстового файла. При интерактивном вводе левый клик по свободному месту холста создаёт вершину с уникальным номером; последовательный клик по двум вершинам инициирует создание ребра с запросом веса (целого числа) через диалоговое окно. Правый клик по вершине или ребру обеспечивает их удаление.

При загрузке из файла поддерживается единый текстовый формат: первая строка содержит два целых числа — количество вершин и количество рёбер. Каждая последующая строка содержит три целых числа: идентификатор начальной вершины, идентификатор конечной вершины и вес ребра. Поскольку алгоритм Краскала работает с неориентированным графом, порядок указания вершин в ребре не имеет значения — программа автоматически приводит ребро к каноническому виду (меньший идентификатор — первый). Наличие петель и кратных рёбер в файле не допускается и обрабатывается как ошибка ввода.

2.2.2. Требования к визуализации

Интерфейс разделён на несколько функциональных областей. Центральная часть — холст для отображения графа, где пользователь создаёт и редактирует вершины и рёбра. Вершины отображаются в виде кругов с подписанным внутри идентификатором; при выборе первой вершины для создания ребра она подсвечивается оранжевым цветом. Рёбра отображаются в виде отрезков с указанием веса в средней точке. Цветовая индикация рёбер отражает состояние алгоритма: серый — ожидает проверки, жёлтый — текущее рассматриваемое, зелёный — добавлено в минимальное остовное дерево, красный — отклонено (образует цикл). Правая часть содержит

панель управления алгоритмом (кнопки «Шаг вперёд», «Шаг назад», «Автозапуск», «Пауза», «Сброс», ползунок скорости), а также мини-карту для обзорного отображения графа. Нижняя часть включает информационную панель со статистикой (текущий вес MST, количество добавленных рёбер, номер шага) и текстовое поле loga для вывода пояснений к каждому действию алгоритма. В верхней части расположена панель инструментов с кнопками загрузки, сохранения и очистки графа.

2.2.3. Уточнение требований после сдачи прототипа

После демонстрации прототипа было подтверждено, что часть кнопок управления на данном этапе может оставаться неактивной (например, пошаговый режим и автозапуск), однако расположение основных функциональных областей должно соответствовать финальной версии приложения. Было принято решение сохранить центральную область с холстом и прокруткой для отображения графа, правую панель с элементами управления алгоритмом, а также нижнюю область для вывода статистики и текстового лога пояснений. Также была зафиксирована необходимость подписывать веса рёбер непосредственно на холсте и отображать цветовую индикацию состояний рёбер (серый — ожидает, жёлтый — рассматривается, зелёный — добавлен в остов, красный — отклонён) для наглядного отслеживания хода выполнения алгоритма.

2.2.4. Уточнение требований после сдачи альфа-версии

После сдачи альфа-версии программы были внесены следующие уточнения, касающиеся интерфейса и логирования. Во-первых, изменён формат вывода сообщений в лог-панели: теперь перед каждым сообщением о рассматриваемом ребре выводится номер шага в виде «Шаг х: », а также убран лишний перенос строки перед записью о добавлении или отклонении ребра, что сделало вывод более компактным и удобным для восприятия. Во-вторых, для единообразия подпись кнопки «Шаг вперед» была заменена на «Шаг вперёд». В-третьих, уточнено поведение кнопок управления: в стартовом состоянии кнопка «Шаг назад» теперь блокируется, а при

достижении последнего шага блокируется кнопка «Шаг вперед». Кроме того, из сопроводительных материалов было исключено утверждение о том, что алгоритм работает только для связных графов, поскольку алгоритм Краскала корректно строит остовный лес для несвязных графов, и данная возможность уже реализована в программе.

2.2.5. Уточнение требований после сдачи бета-версии

Уточнений не было.

2.3. Командная итеративная разработка приложения

Разработка велась в соответствии с утверждённым планом. В процессе работы промежуточные версии программы регулярно демонстрировались руководителю, что позволяло своевременно корректировать требования и дорабатывать функционал. Благодаря чёткому разделению обязанностей и регулярной синхронизации результатов между разработчиками интеграция отдельных модулей проходила без значительных затруднений. Использование системы контроля версий Git позволило эффективно управлять изменениями и фиксировать промежуточные состояния проекта.

2.4. Структуры данных и их методы

Разработанное приложение имеет модульную архитектуру, разделённую на три основных уровня: модели данных, алгоритмическое ядро и пользовательский интерфейс. Такое разделение обеспечивает независимость логики работы алгоритма от способов его визуализации и позволяет легко модифицировать или расширять отдельные компоненты без влияния на остальные части системы. Модели данных хранят структуру графа, алгоритмическое ядро реализует непосредственно алгоритм Краскала с поддержкой пошагового выполнения, а интерфейсный слой отвечает за отображение, интерактивное взаимодействие с пользователем и управление ходом выполнения алгоритма.

Модели данных

Базовым элементом графа является класс **Vertex**, описывающий отдельную вершину. Он содержит целочисленный идентификатор, который

присваивается автоматически при создании и служит для однозначной идентификации вершины в графе, а также координаты x и y , определяющие её положение на холсте. Координаты могут быть заданы как интерактивно — при создании вершины щелчком мыши, так и автоматически — при загрузке графа из файла, когда все вершины равномерно распределяются по окружности для наглядного отображения. Класс `Vertex` реализован как `data-класс Kotlin`, что обеспечивает автоматическую генерацию методов `equals`, `hashCode` и `toString`.

Связи между вершинами моделируются классом **Edge**, который хранит идентификаторы начальной и конечной вершин, а также целочисленный вес ребра. Поскольку алгоритм Краскала работает с неориентированными графами, особое внимание было уделено корректной обработке направленности. Для этого класс переопределяет методы `equals` и `hashCode` таким образом, что ребро приводится к каноническому виду: независимо от того, в каком порядке были указаны вершины при создании, внутренне ребро всегда представляется как пара, где первый идентификатор меньше второго. Это позволяет правильно определять наличие ребра между двумя вершинами независимо от направления, корректно исключать кратные рёбра и обеспечивает стабильную работу хеш-таблиц, используемых для хранения состояний рёбер.

Контейнером для всех элементов графа служит класс **Graph**, который содержит изменяемые списки вершин и рёбер. Он предоставляет удобные геттеры для получения количества вершин и рёбер через свойства `vertexCount` и `edgeCount`, а также метод `hasEdge`, который проверяет существование ребра между двумя вершинами с учётом неориентированности графа. Этот метод используется как при интерактивном создании рёбер для предотвращения дублирования, так и при загрузке графа из файла для валидации входных данных.

Система непересекающихся множеств

Ключевой структурой данных, лежащей в основе алгоритма Краскала,

является система непересекающихся множеств, реализованная классом **Dsu**[5]. Данная структура предназначена для эффективного отслеживания компонентов связности в графе и проверки того, принадлежат ли две вершины одному и тому же множеству. При инициализации класс создаёт массив родителей, где каждый элемент изначально указывает сам на себя, что означает, что каждая вершина является корнем своего собственного множества.

Метод `find` осуществляет поиск корневого элемента множества, содержащего заданную вершину, с применением техники сжатия пути. Это означает, что в процессе поиска все промежуточные вершины на пути к корню перенаправляются непосредственно на корневой элемент, что значительно ускоряет последующие операции и обеспечивает около константное время выполнения в среднем. Метод `union` выполняет объединение двух множеств: сначала находятся корневые элементы для обеих вершин, после чего проверяется их совпадение. Если корни совпадают, вершины уже находятся в одном множестве, и метод возвращает `false`, сигнализируя о том, что добавление ребра между ними создаст цикл. В противном случае один корень становится родительским для другого, множества объединяются, и метод возвращает `true`, что означает возможность безопасного добавления ребра в минимальное остовное дерево.

Алгоритмическое ядро

Основная логика работы алгоритма Краскала сосредоточена в классе **KruskalAlgorithm**. Этот класс содержит ссылку на граф, для которого выполняется построение минимального остовного дерева, список рёбер, вошедших в итоговый остов, и список шагов выполнения, используемый для пошаговой визуализации и поддержки отката.

Метод `run` запускает процесс построения MST[6]. На первом этапе он очищает списки результатов от предыдущих запусков, обеспечивая корректность повторного использования. Затем все рёбра графа сортируются по возрастанию веса с использованием стандартной функции `sortedBy`, что

гарантирует, что алгоритм всегда рассматривает наиболее дешёвые рёбра в первую очередь. После этого создаётся экземпляр структуры DSU, инициализированный количеством вершин графа.

Далее алгоритм последовательно обрабатывает каждое отсортированное ребро. Для текущего ребра вызывается метод `union` структуры DSU, который определяет, принадлежат ли соединяемые вершины разным компонентам связности. Если `union` возвращает `true`, ребро добавляется в список финальных рёбер, а общий вес MST увеличивается на вес этого ребра. В любом случае информация о каждом шаге сохраняется в виде объекта `KruskalStep`, содержащего само ребро, флажок добавления, текущий общий вес и количество уже добавленных рёбер. Эта история шагов позволяет впоследствии отображать пошаговое выполнение алгоритма и возвращаться к предыдущим состояниям.

Класс **`KruskalStep`** представляет собой простой объект-носитель данных, или DTO, который не содержит логики, а служит исключительно для фиксации состояния после каждого шага алгоритма. Благодаря своей структуре он обеспечивает всю необходимую информацию для визуализации: какое ребро рассматривалось, было ли оно добавлено, каков текущий вес построенного остова и сколько рёбер уже в него входит.

Для управления цветовыми состояниями рёбер в процессе визуализации используется перечисление **`EdgeState`**. Оно определяет четыре возможных состояния: `PENDING` — ребро ожидает рассмотрения и отображается серым цветом, `CURRENT` — ребро рассматривается на текущем шаге и подсвечивается жёлтым, `ADDED` — ребро успешно добавлено в минимальное остовное дерево и окрашивается в зелёный цвет, `REJECTED` — ребро отклонено из-за образования цикла и отображается красным. Такая цветовая схема обеспечивает наглядную и интуитивно понятную визуализацию всех этапов работы алгоритма.

Компонент визуализации

Центральным элементом пользовательского интерфейса является

класс **GraphCanvas**, который наследуется от JavaFX-компонента **Canvas** и отвечает за отображение графа на холсте, интерактивное редактирование и визуализацию работы алгоритма. Данный класс объединяет в себе как функции рендеринга, так и обработку событий мыши, что позволяет пользователю взаимодействовать с графом непосредственно на холсте.

Для хранения текущего состояния графа класс использует локальные структуры данных: отображение вершин с их координатами на холсте, список рёбер и карту состояний рёбер, которая сопоставляет каждому ребру его текущее цветовое состояние. Кроме того, класс хранит идентификатор выбранной вершины, используемый при создании рёбер, и счётчик для генерации уникальных идентификаторов новых вершин. Для обеспечения слабой связанности с контроллером класс предоставляет callback-функции, вызываемые при добавлении или удалении вершин и рёбер.

Метод `draw` осуществляет полную перерисовку холста. Он получает графический контекст, очищает область рисования и последовательно отображает все элементы. Сначала рисуются рёбра — для каждого определяется его цветовое состояние из карты `edgeStates`, после чего линия соответствующего цвета проводится между координатами соединяемых вершин. В средней точке каждого ребра отображается его вес. Затем поверх рёбер рисуются вершины — каждая в виде круга с заливкой стального цвета, чёрной обводкой и белым идентификатором внутри. Если вершина выбрана для создания ребра, она подсвечивается оранжевым цветом, что визуально сигнализирует пользователю о текущем режиме редактирования.

Обработчики событий мыши реализуют полный цикл интерактивного редактирования графа. При левом клике по свободному месту холста создаётся новая вершина, при этом система проверяет, чтобы расстояние до существующих вершин было не меньше удвоенного радиуса, предотвращая их наложение и улучшая читаемость графа. При последовательном левом клике по двум вершинам инициируется создание ребра: после выбора второй вершины появляется диалоговое окно с запросом веса, и после ввода числа

ребро добавляется в граф. Правый клик по вершине приводит к её удалению вместе со всеми инцидентными рёбрами, а правый клик по центру ребра удаляет только это ребро. Все изменения немедленно отображаются на холсте благодаря вызову метода `draw`.

Для загрузки готового графа из файла предусмотрен метод `loadGraph`, который принимает объект `Graph`, очищает текущее состояние холста и заполняет его данными из переданной модели. Вершины при этом автоматически расставляются по окружности с центром в середине холста, что обеспечивает равномерное и наглядное распределение даже для больших графов. После загрузки каждому ребру присваивается состояние `PENDING`, и холст перерисовывается.

Для управления подсветкой рёбер в процессе выполнения алгоритма класс предоставляет три метода. Метод `highlightCurrentEdge` подсвечивает заданное ребро жёлтым цветом, визуализируя текущий шаг алгоритма перед принятием решения. Метод `highlightStep` фиксирует результат рассмотрения ребра, изменяя его цвет на зелёный или красный в зависимости от того, было ли оно добавлено в `MST`. Метод `unhighlightStep` возвращает ребро в состояние `PENDING`, что используется при откате шагов. Метод `reset` сбрасывает все рёбра в состояние ожидания, а метод `clear` полностью очищает холст, удаляя все вершины и рёбра.

Главный контроллер приложения

Класс **MainController** является связующим звеном между моделью данных, алгоритмическим ядром и пользовательским интерфейсом. Он управляет всеми компонентами интерфейса, обрабатывает события от кнопок управления и синхронизирует состояние приложения.

При инициализации контроллер настраивает все элементы интерфейса: панель управления с кнопками шагов, автозапуска, паузы, сброса и очистки; ползунков регулировки скорости воспроизведения; информационные панели для отображения статистики; текстовое поле для вывода `loga` выполнения; и мини-карту для обзорного отображения графа. Все элементы связываются с

соответствующими обработчиками событий.

Метод `stepForward` реализует двухфазную логику пошагового выполнения. При первом нажатии на кнопку "Шаг вперёд" текущее рассматриваемое ребро подсвечивается жёлтым цветом через вызов `canvas.highlightCurrentEdge`, и в `log` выводится сообщение о начале рассмотрения ребра. При повторном нажатии происходит фиксация результата: вызывается `canvas.highlightStep`, которая окрашивает ребро в зелёный или красный цвет в зависимости от результата проверки. В `log` добавляется пояснение, было ли ребро добавлено в MST или отклонено, после чего обновляются статистические показатели (вес MST, количество добавленных рёбер, номер шага), и алгоритм переходит к следующему ребру. Такая двухфазная схема позволяет пользователю внимательно следить за каждым этапом принятия решения алгоритмом.

Метод `stepBackward` обеспечивает откат на один шаг. Если в данный момент ребро подсвечено жёлтым цветом, метод просто снимает эту подсветку, возвращая ребро в состояние ожидания. Если же шаг уже был завершён (ребро окрашено в зелёный или красный), метод откатывает последнее зафиксированное действие, возвращая ребро в состояние `PENDING` и восстанавливая предыдущие значения статистики. В обоих случаях в `log` выводится соответствующее сообщение, а состояние интерфейса обновляется.

Метод `startAlgorithm` запускает автоматическое воспроизведение алгоритма в отдельном фоновом потоке, что позволяет основному потоку интерфейса оставаться отзывчивым. Внутри цикла алгоритм последовательно выполняет шаги с задержкой, определяемой значением ползунка скорости. При достижении последнего шага или при нажатии кнопки паузы воспроизведение останавливается. Метод `pauseAlgorithm` прерывает автоматическое воспроизведение, а метод `resetAlgorithm` полностью сбрасывает состояние алгоритма: очищается история шагов, все рёбра возвращаются в серый цвет, статистика обнуляется, а `log` очищается.

По завершении алгоритма вызывается метод `showCompletionMessage`, который формирует итоговый отчёт. Для связных графов выводится список рёбер, вошедших в минимальное остовное дерево, их общий вес и количество. Для несвязных графов программа корректно строит минимальный остовный лес и информирует пользователя о том, что исходный граф не был связным, выводя соответствующее пояснение.

Модули загрузки и сохранения графа

Функция `readGraphFromFile` обеспечивает загрузку структуры графа из текстового файла. На первом этапе выполняется проверка существования файла и его размера — пустые или отсутствующие файлы сразу отклоняются с выбросом исключения. Затем файл читается построчно с использованием `Scanner`. Первая строка должна содержать два целых числа: количество вершин и количество рёбер. После этого читаются строки с описанием рёбер, каждая из которых должна содержать три целых числа: идентификатор начальной вершины, идентификатор конечной вершины и вес. В процессе чтения выполняется комплексная валидация данных: проверяется, что количество фактически прочитанных рёбер совпадает с заявленным в заголовке; контролируется отсутствие петель, то есть рёбер, соединяющих вершину с самой собой; проверяется, что все индексы вершин находятся в допустимом диапазоне от 0 до $V-1$; обнаруживаются изолированные вершины, заявленные в заголовке, но не фигурирующие ни в одном ребре. При обнаружении любой ошибки выбрасывается соответствующее исключение с информативным сообщением, которое перехватывается на уровне контроллера и отображается пользователю в виде диалогового окна. Такая тщательная проверка гарантирует, что в алгоритм поступают только корректные данные.

Функция `saveGraphToFile` выполняет обратную операцию — запись текущего графа в текстовый файл. В первой строке сохраняется количество вершин и рёбер, после чего для каждого ребра записываются его начальная вершина, конечная вершина и вес. Формат сохранения полностью

соответствует формату загрузки, что обеспечивает совместимость и позволяет сохранять созданные графы для последующего использования.

2.5. Тестирование

2.5.1. Тестирование интерфейса

Интерфейс проверялся при начальном состоянии, в процессе выполнения алгоритма и после его завершения. Было подтверждено, что кнопки «Шаг назад» и «Шаг вперёд» активируются только при наличии соответствующего шага, а при достижении крайних состояний (начало или конец алгоритма) они автоматически блокируются. Режим автозапуска корректно блокирует несовместимые элементы управления (кнопки редактирования графа, загрузки и сохранения), предотвращая внесение изменений в граф во время работы алгоритма. Кнопка «Пауза» доступна только в процессе автозапуска, а изменение графа (добавление, удаление вершин и рёбер) приводит к автоматическому сбросу устаревшей визуализации и очистке истории шагов.

Проверена читаемость подписей вершин (идентификаторы внутри кругов) и весов рёбер (числа в средней точке отрезка). Текстовая область лога корректно переносит длинные сообщения по словам и снабжена вертикальной и горизонтальной полосами прокрутки, что обеспечивает удобство чтения даже при большом количестве шагов. Холст с графом размещён внутри контейнера `ScrollPane`, что позволяет перемещаться по увеличенному изображению и работать с графами, размер которых превышает видимую область окна.

2.5.2. Тестирование алгоритма

Для простого взвешенного связного графа была выполнена визуализации алгоритма Краскала, в результате которого было построено минимальное остовное дерево с общим весом, соответствующим правильному значению. Корректность проверена для каждого добавленного ребра: все они не образуют циклов, а суммарный вес остова является минимальным. Для

несвязного графа алгоритм успешно построил минимальный остовный лес (рис. 2), включив все вершины в соответствующие компоненты связности.

Кроме того, для графа, содержащего ребро с отрицательным весом, алгоритм продолжает работать корректно, поскольку жадная стратегия Краскала не требует неотрицательности весов (в отличие, например, от алгоритма Дейкстры)(рис. 3).

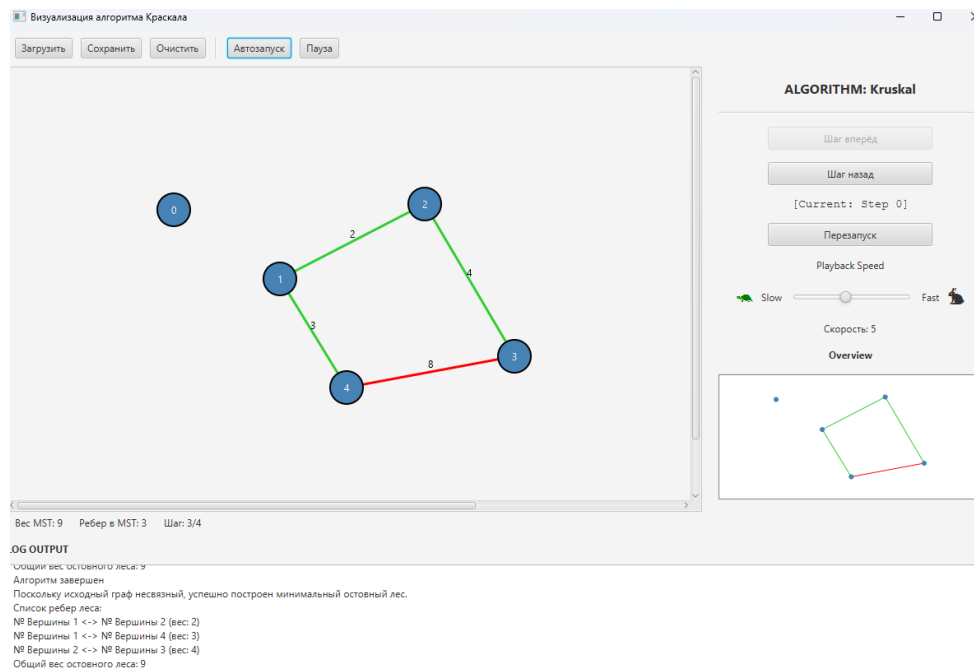


Рисунок 2 – Минимальный остовный лес

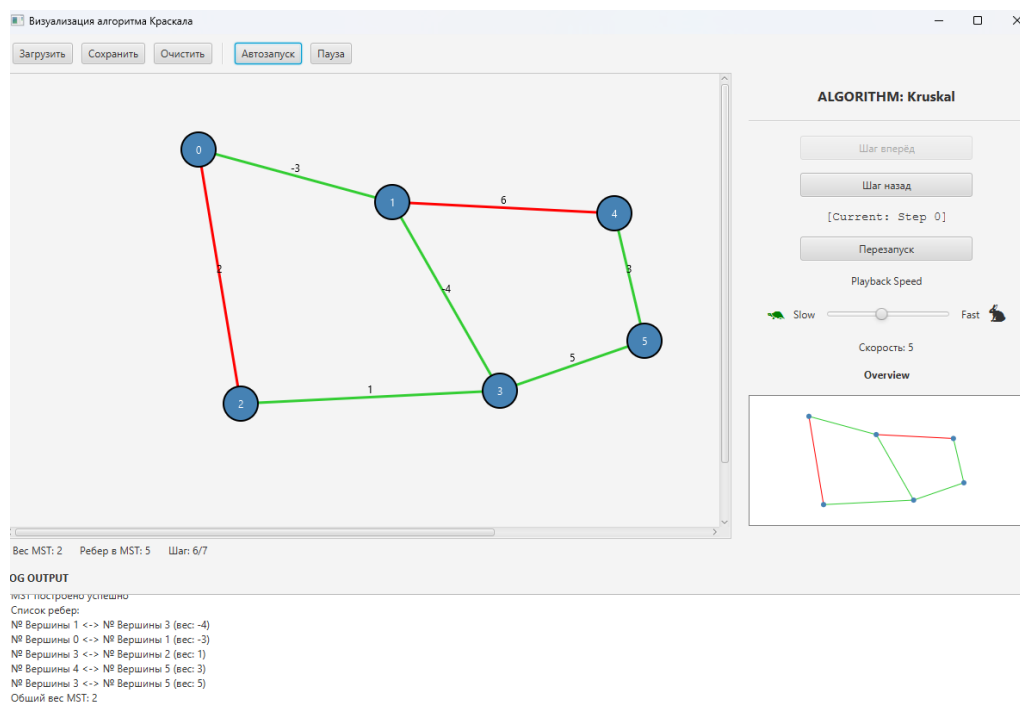


Рисунок 3 – МСТ с отрицательными весами

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

Технология	Назначение в проекте
Kotlin 1.9+	Модель данных (Vertex, Edge, Graph), реализация алгоритма Краскала и структуры DSU, логика пошагового выполнения, сервисы ввода-вывода, обработка исключений
JavaFX	Главное окно (Stage, Scene), компоновка элементов (BorderPane, HBox, VBox, ScrollPane), холст (Canvas, GraphicsContext) для отрисовки графа, элементы управления (Button, Label, Slider, TextArea), диалоговые окна (Alert, TextInputDialog, FileChooser)
GraphicsContext (JavaFX)	Отрисовка вершин (круги с подписями), рёбер (отрезки с весами), цветовая индикация состояний (серый, жёлтый, зелёный, красный), работа с текстом и геометрическими примитивами
Kotlin Collections	MutableList, MutableMap, функции работы с коллекциями (sortedBy, distinct, flatMap, any, filter), неизменяемые данные
java.io (стандартная библиотека Kotlin/Java)	Чтение и запись текстовых файлов (Scanner, FileWriter), проверка существования файла, валидация формата входных данных

Git и GitHub	Совместная разработка, история изменений, ветки, коммиты, синхронизация результатов между разработчиками
JAR (Java Archive)	Доставка исполнимой версии приложения, запускаемой вне среды разработки
Stepik и официальная документация Kotlin/JavaFX	Самостоятельное изучение языка Kotlin, основ JavaFX и принципов построения графических интерфейсов

Таблица 5 – Используемые технологии и средства

Для построения интерфейса использованы классы JavaFX: Application, Stage, Scene, BorderPane, HBox, VBox, ScrollPane, Canvas, GraphicsContext, Button, Label, Slider, TextArea, Alert, TextInputDialog, FileChooser. Автоматическое воспроизведение алгоритма реализовано в отдельном фоновом потоке с использованием Thread и Platform.runLater для безопасного обновления элементов интерфейса из фонового потока, что исключает зависания и обеспечивает отзывчивость интерфейса.

Для сборки проекта использовалась система сборки Gradle. Сценарий сборки компилирует исходные файлы Kotlin, создаёт исполнимый JAR-архив с указанием главного класса ui.AppLauncher (или ui.MainApp). Полученный файл (например, kruskal-visualizer.jar) запускается командой `java -jar` и не требует среды разработки, что обеспечивает возможность его использования в учебных классах и на компьютерах студентов без установленной IDE.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

ЗАКЛЮЧЕНИЕ

В ходе учебной практики была выполнена командная разработка программного продукта — визуализатора алгоритма Краскала для построения минимального остовного дерева на языке Kotlin с использованием библиотеки JavaFX. Полностью решены все поставленные задачи: изучены основы JavaFX и принципы построения графических интерфейсов, разработана архитектура приложения с разделением на модель данных, компоненты визуализации и алгоритмическое ядро, реализован алгоритм Краскала на основе системы непересекающихся множеств (DSU) с поддержкой пошагового выполнения (включая двухфазную подсветку рёбер), отката и сброса, создан интерактивный редактор графа с управлением через мышь (добавление вершин, создание взвешенных рёбер, удаление элементов), реализованы механизмы загрузки и сохранения графа в текстовом формате с комплексной валидацией данных, проведена ручная проверка на различных типах графов (связные, несвязные, с петлями и кратными рёбрами, с различным количеством вершин), а также проверена корректность работы интерфейса во всех режимах — ошибок и непредвиденного поведения не выявлено.

Разработанное приложение полностью соответствует утверждённой спецификации и может использоваться в учебном процессе для наглядной демонстрации работы алгоритма Краскала, а также для изучения принципов работы жадных алгоритмов и структуры данных «система непересекающихся множеств». Полученные навыки объектно-ориентированного программирования на Kotlin, разработки графических интерфейсов на JavaFX и командной работы с использованием системы контроля версий будут полезны в дальнейшей профессиональной деятельности.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Теория графов[Электронный ресурс].
URL: <https://habr.com/ru/companies/otus/articles/568026/> (дата обращения: 14.07.2026)
2. JavaFX [Электронный ресурс].
URL: <https://openjfx.io/openjfx-docs/> (дата обращения: 13.07.2026).
3. Онлайн-курс Kotlin[Электронный ресурс].
URL: <https://stepik.org/course/5448/syllabus> (дата обращения: 08.07.2026).
4. Алгоритм Краскала[Электронный ресурс].
URL: <https://habr.com/ru/articles/569444/> (дата обращения 14.07.2026)
5. DSU[Электронный ресурс].
URL: <http://e-maxx.ru/algo/dsu> (дата обращения 14.07.2026)
6. MST[Электронный ресурс].
URL: <https://algorithmica.org/ru/mst> (дата обращения 14.07.2026)

ПРИЛОЖЕНИЕ А

Представляется финальный вид пользовательского интерфейса приложения (рис. 4).

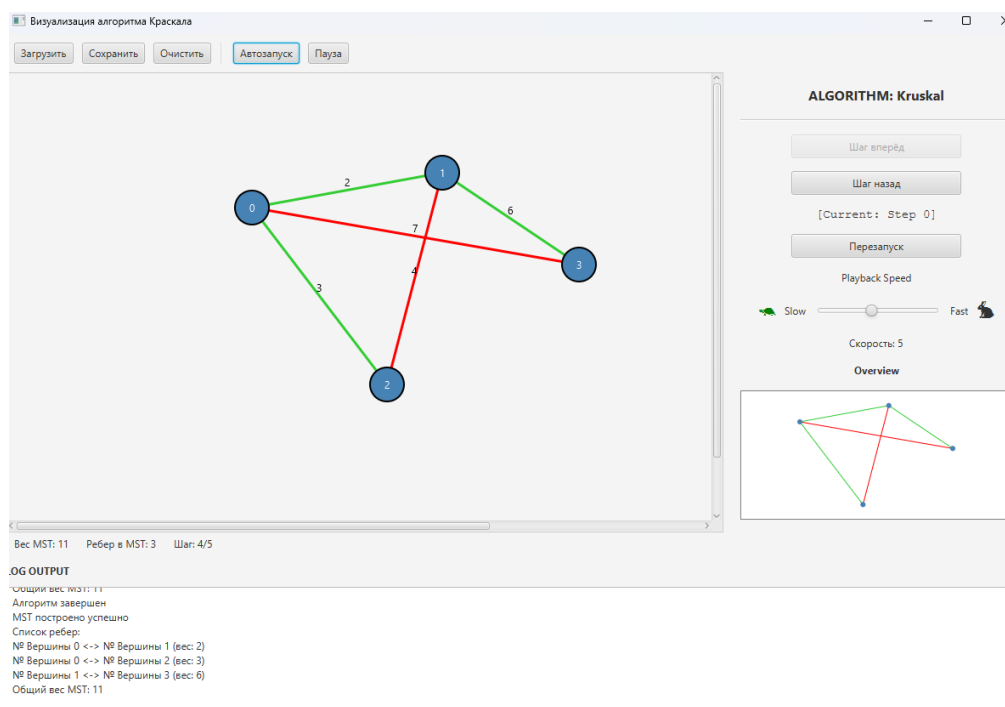


Рисунок 4 – Интерфейс программы